CSP-J 质量抽检模拟试卷（含答案与解析）

考试时间：120 分钟　满分：100 分　生成时间：2026/05/07 08:39:55

姓名：_____ 班级：_____ 得分：_____

请在答题区域作答；单项选择题、阅读程序题和完善程序题均为单选。除特别说明外，程序均按 C++17 标准理解。

抽样来源：papers/2026 已入库题库文件仓库；蓝图版本：config/blueprint.ts v1；抽样种子：20260507034045

一、单项选择题（共 15 题，每题 2 分，共 30 分）

1. （2 分）在C++中，表达式 (3 > 2) + (2 > 3) 的值是多少？

A. 0
B. 1
C. 2
D. true

答案：B

解析：

C++中，关系运算符返回bool类型，true转换为整数1，false转换为0。(3 > 2)为true即1，(2 > 3)为false即0，1+0=1。

2. （2 分）关于计算机中的“位”（bit），以下说法正确的是：

A. 一个位是计算机中最小的存储单位，只能表示0或1
B. 一个字节由4个位组成
C. 一个位可以表示十进制的0到9
D. 所有计算机的数据类型都以位为单位存储

答案：A

解析：

位（bit）是二进制数的最小单位，只有0和1两种状态。一个字节由8个位组成，而不是4个。位不能直接表示十进制的0到9。虽然底层数据以位存储，但通常以字节为单位寻址和访问。

3. （2 分）在计算机中，一个ASCII字符占用几个字节？

A. 1
B. 2
C. 4
D. 8

答案：A

解析：

标准ASCII码使用7位表示一个字符，但在计算机存储中通常占用1个字节（8位）。因此答案为1字节。

4. （2 分）在 C++ 中，`vector<int> v = {1, 2, 3, 4, 5};`，执行 `v.erase(v.begin() + 2);` 后，`v[2]` 的值是？

A. 3
B. 4
C. 5
D. 未定义

答案：B

解析：

`erase` 删除索引 2（即第三个元素，值为 3）。删除后，后面的元素向前移动：原 {1,2,3,4,5} 变为 {1,2,4,5}。因此 `v[2]` 是第四个元素，值为 4。

5. （2 分）下面这段代码执行后，输出的结果是？

```
#include <iostream>
using namespace std;
int main() {
    int a = 1, b = 2, c = 3;
    cout << (a < b < c) << endl;
    return 0;
}
```

A. 0

B. 1

C. 2

D. 3

答案：B

解析：

表达式`a < b < c`在C++中从左向右结合。先计算`a < b`，结果为真（1）。然后计算`1 < c`，即`1 < 3`，结果为真（1）。因此输出1。

6. （2 分）C++ 中，关于结构体（struct）和类（class）的区别，以下哪个说法是正确的？

A. struct 中的成员默认是 public 的，class 中的成员默认是 private 的

B. struct 不能包含成员函数，class 可以

C. struct 不能继承，class 可以继承

D. struct 和 class 在 C++ 中没有区别

答案：A

解析：

在 C++ 中，struct 和 class 的唯一区别在于默认访问权限：struct 成员默认为 public，class 成员默认为 private。两者都可以包含成员函数、构造函数、析构函数，都支持继承、多态等面向对象特性。选项 B、C 错误，选项 D 不正确。

7. （2 分）以下C++代码片段的输出是什么？

```
#include <iostream>
using namespace std;
int main() {
    int a = 5, b = 3;
    cout << (a > b ? a++ : b++) << endl;
    cout << a << " " << b << endl;
    return 0;
}
```

A. 5
   6 3

B. 5
   5 4

C. 5
   6 4

D. 6
   6 4

答案：A

解析：

第一行输出条件表达式的结果。由于a > b成立，执行a++，先返回a的当前值5，然后a自增为6，b不变，所以输出5。第二行输出a和b的值，此时a为6，b为3，输出"6 3"。注意条件表达式是短路求值，只执行一个分支。因此最终输出为两行：第一行5，第二行6 3。

8. （2 分）以下C++代码的输出是什么？

```
#include <iostream>
using namespace std;
int main() {
    int x = 5;
    int y = ++x * 2;
```

```
    int z = x++ + 3;
    cout << x << ' ' << y << ' ' << z << endl;
    return 0;
}
```

A. 7 12 9

B. 7 12 8

C. 6 12 8

D. 6 10 8

答案：A

解析：

`++x` 是先自增后使用，所以 `x` 变为6，`y = 6 * 2 = 12`。然后 `x++` 是先使用后自增，`z = 6 + 3 = 9`，之后 `x` 变为7。因此输出为 `7 12 9`。

9. （2 分）Q0435（CSP-J/easy）：对长度为 11 的有序数组做标准二分查找，最多需要比较多少次？

A. 2

B. 3

C. 5

D. 4

答案：D

解析：

二分查找每次将区间约减半，最坏比较次数为 floor(log2(11)) + 1 = 4。

10. （2 分）以下哪种排序算法是稳定的？

A. 快速排序

B. 选择排序

C. 堆排序

D. 归并排序

答案：D

解析：

归并排序在合并过程中保持相等元素的相对顺序，是稳定的。快速排序、选择排序和堆排序都是不稳定的。

11. （2 分）在对数组 [5, 2, 4, 6, 1, 3] 进行插入排序（升序）的过程中，当处理到元素 1 时（即当前已排序部分为 [2,4,5,6]，待插入元素为 1），需要进行多少次元素移动（后移）操作？

A. 3

B. 4

C. 5

D. 2

答案：B

解析：

插入排序：已排序部分 [2,4,5,6]，插入1，需要将2,4,5,6都向后移动一位，共4次移动。

12. （2 分）一组记录的关键码为 (46, 79, 56, 38, 40, 84)，采用堆排序（大根堆）的初始建堆阶段，得到的初始堆是（ ）。

A. 84, 79, 56, 38, 40, 46

B. 84, 79, 46, 38, 40, 56

C. 84, 56, 79, 40, 46, 38

D. 84, 79, 38, 46, 40, 56

答案：A

解析：

初始序列：46,79,56,38,40,84。从最后一个非叶节点56（i=2）开始调整。i=2: 56与84比，小于84，交换56和84得 46,79,84,38,40,56。i=1: 79与子84,38比，84>79，交换79和84得 46,84,79,38,40,56；再向下调整46？i=0: 46与子 84,79比，84最大，交换46和84得 84,46,79,38,40,56；再调整46：与子38,40比，46>40和38，不交换。最终堆：84,46,79,38,40,56 ？ 但选项A是84,79,56,38,40,46，不一致。重新建堆：采用标准建堆过程，自底向上。数组索引0开始，6个元素，最后一个非叶节点索引 (6/2)-1=2，元素56。调整56为根的子树：子为索引5元素84，56<84，交换得 46,79,84,38,40,56。然后i=1（79）：子为3的38和4的40，79大于38和40，不调整。i=0（46）：子为1的79和2的84，84最大，46<84，交换46和84得 84,79,46,38,40,56；再调整46为根的子树（索引2）：子为56，46<56，交换得 84,79,56,38,40,46。此时堆为84,79,56,38,40,46，对应选项A。

13. （2 分）C++ priority_queue<int> 默认弹出的元素是？

A. 最小元素

B. 最大元素

C. 最早插入元素

D. 最晚插入元素

答案：B

解析：

默认 priority_queue 是大根堆，top 为最大值。

14. （2 分）已知正整数 n 的二进制表示为 11010，则 n 的十进制值为（ ）。

A. 26

B. 24

C. 22

D. 28

答案：A

解析：

二进制 11010 转换为十进制：1×2^4 + 1×2^3 + 0×2^2 + 1×2^1 + 0×2^0 = 16 + 8 + 0 + 2 + 0 = 26。

15. （2 分）在C++中，表达式 sizeof(char) 的值通常是多少？

A. 0

B. 1

C. 2

D. 4

答案：B

解析：

C++标准规定sizeof(char)恒为1字节。

二、阅读程序题（共 3 题，每题 5 小题，共 40 分；小题分值见题内标注）

16. （本题 14 分）阅读以下C++17程序，分析其执行过程。

```
#include <iostream>
using namespace std;
int main() {
    int a = 5;
    int b = 10;
    int c = 0;
    c = (a > b) ? a : b;
    int d = (a % 2 == 0) ? (a * 2) : (b * 2);
    cout << "c=" << c << endl;
    cout << "d=" << d << endl;
    int e = 0;
    for (int i = 1; i <= 3; ++i) {
        e += i;
    }
    cout << "e=" << e << endl;
```

```
    return 0;
}
```

```
c=10
d=20
e=6
```

16.1 （3 分）程序输出的第一行是什么？

A. c=5

B. c=10

C. c=0

D. c=15

答案：B

16.2 （3 分）程序输出的第二行是什么？

A. d=5

B. d=10

C. d=20

D. d=0

答案：C

16.3 （3 分）程序输出的第三行是什么？

A. e=6

B. e=3

C. e=1

D. e=0

答案：A

16.4 （3 分）在程序执行到return 0;之前，变量d的最终值是多少？

A. 10

B. 20

C. 5

D. 15

答案：B

16.5 （2 分）程序中循环的执行次数是多少？

A. 1

B. 2

C. 3

D. 4

答案：C

解析：

1. 变量a为5，b为10，条件a > b为false，因此三元运算符(a > b) ? a : b的结果为b，即10，赋值给c。第一行输出c=10。

2. 变量a为5，条件a % 2 == 0（5除以2余1）为false，因此执行(b * 2)，即10 * 2 = 20，赋值给d。第二行输出d=20。
3. 变量e初始为0。循环从i=1到3，每次将i加到e上：第一次e=0+1=1；第二次e=1+2=3；第三次e=3+3=6。循环结束后e=6，第三行输出e=6。
4. 根据第二问分析，d在计算后赋值为20，且之后没有被修改，因此最终值为20。
5. for循环的初始化i=1，条件i<=3，每次迭代后i递增1。当i=1、2、3时执行循环体，共3次。i=4时条件不满足，循环结束。

17. （本题 13 分）阅读以下程序，回答问题。

```
#include <iostream>
#include <vector>
```

```
using namespace std;

int main() {
    int n = 5;
    vector<int> a = {3, 1, 4, 1, 5};
    int cnt = 0;
    for (int i = 0; i < n; ++i) {
        bool ok = true;
        for (int j = 0; j < i; ++j) {
            if (a[j] > a[i]) {
                ok = false;
                break;
            }
        }
        if (ok) {
            cnt++;
        }
    }
    cout << cnt << endl;
    return 0;
}
```

17.1 （3 分）程序运行后，变量 cnt 的最终值是：

A. 2

B. 3

C. 4

D. 5

答案：B

17.2 （3 分）程序运行后，输出到屏幕的内容是：

A. 2

B. 3

C. 4

D. 5

答案：B

17.3 （3 分）在 i=2 的循环中，内层循环 j 的取值范围是：

A. 0

B. 0 到 1

C. 0 到 2

D. 0 到 3

答案：B

17.4 （2 分）如果数组 a 变为 {1, 2, 3, 4, 5}，程序运行后输出会是：

A. 1

B. 3

C. 5

D. 0

答案：C

17.5 （2 分）该算法的时间复杂度是：

A. O(1)

B. O(n)

C. O(n log n)

D. O(n^2)

答案：D

解析：

1. 程序统计数组中，从索引0到当前索引i，a[i]大于等于之前所有元素的个数。数组为{3, 1, 4, 1, 5}。遍历过程：i=0，之前无元素，ok为真，cnt=1；i=1，比较a[0]=3 > a[1]=1，ok为假，cnt不变；i=2，比较a[0]=3 <= 4且a[1]=1 <= 4，ok为真，cnt=2；i=3，比较a[0]=3 > 1，ok为假；i=4，比较a[0]=3 <=5，a[1]=1 <=5，a[2]=4 <=5，a[3]=1 <=5，ok为真，cnt=3。最终cnt=3。
2. 程序最后输出cnt的值。根据第一问的分析，cnt最终为3，因此输出为3。
3. 当 i=2 时，内层循环条件为 j < i，即 j < 2，所以 j 的取值为 0 和 1。
4. 数组完全递增，每个元素都大于等于之前的所有元素。因此每个 i 都满足 ok 条件，cnt 从 0 增加到 5。最终输出为 5。
5. 外层循环 n 次，内层循环最多 i 次，总循环次数约为 n(n-1)/2，因此时间复杂度为 O(n^2)。

18. （本题 13 分）阅读以下 C++ 程序，假设输入为 6，回答下列问题。

```
#include <iostream>
#include <stack>
using namespace std;

int main() {
    int n;
    cin >> n;
    stack<int> s;
    for (int i = 1; i <= n; ++i) {
        if (i % 2 == 1)
            s.push(i);
        else {
            if (!s.empty()) {
                s.pop();
            }
            s.push(i * 2);
        }
    }
    int cnt = 0;
    while (!s.empty()) {
        cnt += s.top();
        s.pop();
    }
    cout << cnt << endl;
    cout << s.size() << endl;
    return 0;
}
```

18.1 （3 分）程序第一行输出的值是多少？

A. 24

B. 20

C. 18

D. 30

答案：A

18.2 （3 分）程序第二行输出的值是多少？

A. 0

B. 1

C. 2

D. 3

答案：A

18.3 （3 分）如果将输入改为 5，程序第一行输出的值是多少？

A. 15

B. 17
C. 19
D. 13
答案：B

18.4 （2 分）如果将输入改为 7，程序第二行输出的值是多少？

A. 0
B. 1
C. 2
D. 4
答案：A

18.5 （2 分）对于任意正整数输入 n，栈中最多可能存放的元素个数是多少？

A. ⌊n/2⌋
B. ⌊n/2⌋ + 1
C. n
D. ⌈n/2⌉
答案：B

解析：

1. 输入 6，循环过程：i=1(奇) push 1，栈 [1]；i=2(偶) pop 1(栈变[])，push 4，栈 [4]；i=3 push 3，栈 [4,3]；i=4 pop 3(栈 [4])，push 8，栈 [4,8]；i=5 push 5，栈 [4,8,5]；i=6 pop 5(栈 [4,8])，push 12，栈 [4,8,12]。while 循环依次弹出 12、8、4 并累加：12+8=20，20+4=24。输出 24，对应选项 A。
2. while 循环结束后栈为空，s.size() 返回 0，输出 0。
3. n=5 时，最终栈为 [4,8,5]（栈顶5）。依次弹出 5+8+4=17。选项 B 为 17 正确。
4. n=7 最终栈为 [4,8,12,7]？过程：i=1 push1 [1]；i=2 pop1 push4 [4]；i=3 push3 [4,3]；i=4 pop3 push8 [4,8]；i=5 push5 [4,8,5]；i=6 pop5 push12 [4,8,12]；i=7 push7 [4,8,12,7]。累加后 s.empty() 为 true，size=0，输出 0。
5. 分析：每处理一对（奇数+偶数），栈大小先因奇数+1，后因偶数 pop 再 push 保持不变，所以每对净增 0，但初始奇数单独增加 1。因此栈最大元素个数等于读取的奇数个数，即 ⌈n/2⌉。如 n=6 最大 3，n=7 最大 4。选项 A 符合。

三、完善程序题（共 2 题，共 30 分；各空分值见题内标注）

19. （本题 15 分，共 4 空）以下程序读入一个正整数 n，接着读入 n 个整数，输出其中第一个大于所有前面数之和的数。如果不存在这样的数，则输出 -1。请将程序中空白处填写完整。

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cin >> n;
    int sum = 0;
    int num;
    bool found = false;
    for (int i = 0; i < n; i++) {
        cin >> num;
        if (BLANK1) {
            cout << num << endl;
            BLANK2
        }
        BLANK3
    }
    if (BLANK4) {
        cout << -1 << endl;
    }
    return 0;
}
```

样例输入：

```
5
1 2 5 10 20
```

样例输出：

```
1
```

空 1（BLANK1，4 分）

A. num > sum && !found

B. num > sum || found

C. num > sum

D. num >= sum

答案：A

空 2（BLANK2，4 分）

A. found = true;

B. found = false;

C. sum += num;

D. break;

答案：A

空 3（BLANK3，4 分）

A. sum += num;

B. sum = num;

C. num += sum;

D. continue;

答案：A

空 4（BLANK4，3 分）

A. !found

B. found

C. num <= sum

D. found == true

答案：A

解析：

1. 我们需要第一个大于前面所有数之和的数，因此当num > sum且尚未找到(found为false)时输出并标记。选项A正确。选项B使用或运算，一旦found为true仍可能进入（如果num>sum），不符合"第一个"；选项C没有检查found，可能输出多个；选项D不等价于严格大于。
2. 找到后应标记found为true，选项A正确。选项B设false会重置状态；选项C累加错误；选项D跳出循环，但之后无法继续累加后面数的和（尽管题目仅要求第一个，但跳出可能遗漏累加，不过此程序累加在BLANK3，先判断还是先累加需一致，此处我们设计为先判断后累加，详情见完整程序。若用break，则跳过BLANK3的累加，但此时已经输出，且后续数不再处理，不影响第一个的判定，但程序逻辑上BLANK3在判断内还是判断外？分析：在给出的框架中，BLANK1在if内，BLANK2在if内，BLANK3在if外循环内。若BLANK2为break，则离开循环，BLANK3不会执行，而sum未更新此num，但此num已被找到并输出，后续数不再扫描，但是否影响？题目要求第一个大于前面所有数之和的数，找到后即可停止扫描，所以用break也可以。但是为了与BLANK4的found判断配合，found是必须的，否则break后found仍为false，BLANK4会输出-1。所以必须设置found。因此break不能代替found=true。且选项D仅break，没有设置found，导致逻辑错误。故答案选A。
3. 每次读入一个数后，无论是否找到目标，都应将其加入总和，以备后续判断。选A。选B会用num替换总和，错误；选C改变num值且逻辑不对；选D直接跳过累加，导致sum始终为0。
4. 循环结束后，如果从未找到，found为false，应输出-1。选项A正确。选项B在找到时输出-1，反了；选项C此时num为最后一个数，其与sum的关系不能代表是否找到；选项D等价于B，错误。

参考完整代码：

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cin >> n;
    int sum = 0;
```

```
    int num;
    bool found = false;
    for (int i = 0; i < n; i++) {
        cin >> num;
        if (num > sum && !found) {
            cout << num << endl;
            found = true;
        }
        sum += num;
    }
    if (!found) {
        cout << -1 << endl;
    }
    return 0;
}
```

20. （本题 15 分，共 5 空）以下代码模拟了一个简单的栈操作。栈使用数组 `s` 存储，`top` 指向栈顶元素的位置（-1 表示空栈）。程序依次执行入栈、出栈和取栈顶操作，并输出结果。请补全代码。

```
#include <iostream>
using namespace std;

int main() {
    const int MAXN = 100;
    int s[MAXN];
    int top = -1;

    // 入栈操作
    top = BLANK1;
    s[top] = 10;
    top = BLANK2;
    s[top] = 20;
    top = BLANK3;
    s[top] = 30;

    // 出栈操作
    cout << s[top] << " ";
    top = BLANK4;
    cout << s[top] << " ";
    top = BLANK5;

    // 取栈顶操作（不弹出）
    cout << s[top] << endl;
    return 0;
}
```

样例输入：

样例输出：

```
30 20 10
```

空 1（BLANK1，3 分）

A. -1

B. 0

C. top + 1

D. top

答案：B

空 2（BLANK2，3 分）

A. top - 1

B. top

C. top + 1

D. 0

答案：C

空 3（BLANK3，3 分）

A. top + 2
B. top + 1
C. top - 1
D. 1

答案：B

空 4（BLANK4，3 分）

A. top + 1
B. top - 1
C. top
D. -1

答案：B

空 5（BLANK5，3 分）

A. top - 1
B. top + 1
C. top
D. -2

答案：A

解析：

1. 初始时栈为空，top = -1。第一次入栈时，栈顶位置应为 0，即 top = 0，然后 s[top] = 10。若选 A (-1)，则 s[-1] 越界；选 C (top+1) 与 B 等价，但题目要求每个空白填一个具体操作，B 更直接；选 D (top) 仍为 -1，同样越界。
2. 栈中已有元素 10（top=0），第二次入栈应将栈顶指针加 1，即 top = top + 1，然后 s[top] = 20。若选 A，top 变为 -1，覆盖 10；选 B 或 D 不会增加栈顶位置。
3. 当前 top=1（栈中有 10,20），第三次入栈同样需 top = top + 1，使 top=2，然后 s[top] = 30。选 A 会跳过一个位置；选 C 会回退；选 D 会覆盖 10 的位置。
4. 出栈操作是弹出栈顶元素并移动栈顶指针。第一次 cout 输出 s[top]（30）后，top 应减 1 以弹出 30，即 top = top - 1。若选 A 则栈顶指针错误增大；选 C 或 D 不会正确弹出。
5. 第二次 cout 输出 s[top]（20）后，需再次弹出，top 应减 1，即 top = top - 1，使 top 从 1 变为 0。之后取栈顶时 s[top] 应为 10。选 B 会增加指针；选 C 不会弹出；选 D 会超出栈范围。

参考完整代码：

```
#include <iostream>
using namespace std;

int main() {
    const int MAXN = 100;
    int s[MAXN];
    int top = -1;

    // 入栈操作
    top = 0;
    s[top] = 10;
    top = top + 1;
    s[top] = 20;
    top = top + 1;
    s[top] = 30;

    // 出栈操作
    cout << s[top] << " ";
    top = top - 1;
    cout << s[top] << " ";
    top = top - 1;

    // 取栈顶操作（不弹出）
    cout << s[top] << endl;
    return 0;
}
```